Laboratorul 6. GitHub, pas cu pas

# Arsenalul sub lupă

fișă de laborator · 90 min · se lucrează individual, la terminal și în browser

> CE FACEȚI ASTĂZI. Învățați GitHub de la zero, pas cu pas. Fiecare pas are comanda exactă și ce trebuie să apară pe ecran. La final aveți două depozite publice de exercițiu și știți să salvați, să trimiteți, să aduceți înapoi și să citiți istoria unui proiect. La final pregătești raportul și predarea în depozitul privat al disciplinei.

> CUM LUCRAȚI LA LABORATORUL 6. Faceți pașii în ordine, fără să săriți. După fiecare comandă, comparați ce vedeți cu caseta verde „Pe ecran trebuie să vezi". Dacă nu seamănă, vă opriți și citiți tabelul de erori de la A12 sau chemați asistentul. Nu treceți mai departe cu un pas care nu a ieșit.

## LABORATORUL 6. GitHub, pas cu pas

Obiectiv: un cont GitHub, git configurat pe calculatorul tău și două depozite publice de exercițiu, exercitiu-git și exercitiu-2, cu mai multe commit-uri trimise pe GitHub. În cuvinte simple: git ține istoria proiectului pe calculatorul tău, iar GitHub ține o copie a ei pe internet.

| Cuvântul | Ce înseamnă |
|---|---|
| depozit (repository) | folderul proiectului, împreună cu toată istoria lui |
| commit | o fotografie a proiectului la un moment dat, cu un mesaj |
| push | trimiți commit-urile tale de pe calculator pe GitHub |
| pull | aduci de pe GitHub pe calculator ce s-a schimbat acolo |
| clone | descarci prima dată un depozit de pe GitHub |
| origin | numele scurt al copiei de pe GitHub |

### A1. Contul GitHub (10 min)

Dacă ai deja cont GitHub, autentifică-te și sari la A2.

1. Deschide în browser github.com și apasă Sign up.
2. Scrie adresa de e-mail pe care o citești zilnic (poate fi cea de student).
3. Alege o parolă și un nume de utilizator (username). Numele apare în toate linkurile tale: alege ceva profesional, de exemplu ion-popescu sau ipopescu, nu xX_hacker_Xx.
4. Rezolvă verificarea și confirmă codul primit pe e-mail.
5. Notează-ți numele de utilizator: îl folosești imediat.

**De observat.** GitHub îți poate cere mai târziu autentificarea în doi pași (2FA). Activeaz-o când ți-o cere, cu o aplicație de autentificare pe telefon: e același principiu pe care îl recomanzi oricui ca specialist în securitate.

### A2. Git pe calculator: verifici și te prezinți (10 min)

Deschide terminalul (PowerShell pe Windows sau terminalul pe Linux) și verifică dacă git e instalat.

```
git --version
```

**Pe ecran trebuie să vezi:**

```
git version 2.53.0.windows.1      (numărul poate fi altul, nu contează)
```

*Dacă vezi „git is not recognized" sau „command not found", git nu e instalat: pe Windows rulează winget install Git.Git, închide și redeschide terminalul; pe Linux, sudo apt install git.*

Acum spui o singură dată cine ești. Pune numele tău real și EXACT adresa de e-mail din contul GitHub: așa GitHub leagă commit-urile de contul tău.

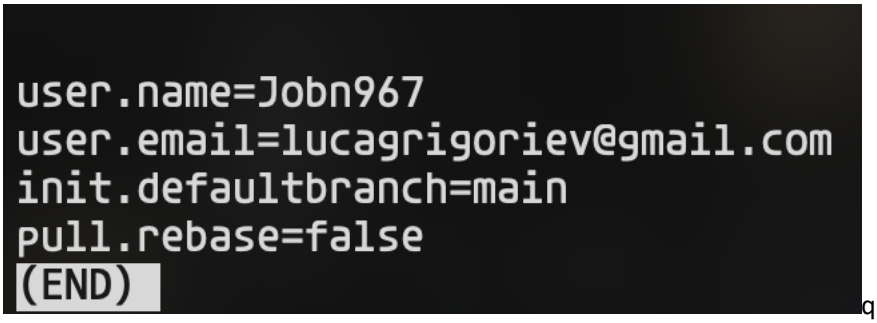
```
git config --global user.name "Ion Popescu"
git config --global user.email "adresa-ta@exemplu.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

Verifici ce ai setat:

```
git config --global --list
```

**Pe ecran trebuie să vezi:**

```
user.name=Ion Popescu
user.email=adresa-ta@exemplu.com
init.defaultbranch=main
pull.rebase=false
```



q

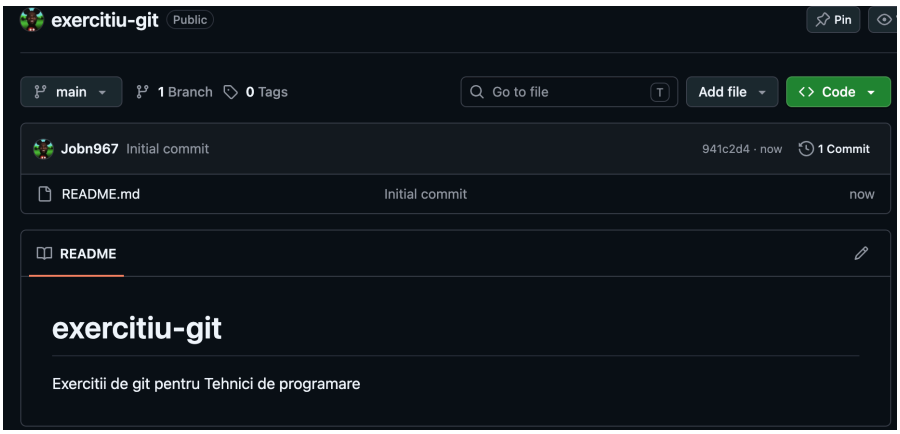
**Ce fac ultimele două linii.** `init.defaultBranch main` numește ramura principală „main", ca pe GitHub. `pull.rebase false` îi spune lui git cum să unească schimbările la pull, ca să nu te întrebe mai târziu.

## A3. Primul depozit, creat pe GitHub (8 min)

În browser, pe GitHub:

1. Sus în dreapta apasă semnul + și alege New repository.
2. La Repository name scrie: exercitiu-git
3. La Description (opțional): Exercitii de git pentru Tehnici de programare
4. Alege Public.
5. Bifează Add a README file.
6. Apasă Create repository.

Ești acum pe pagina depozitului, la adresa https://github.com/<numele-tău>/exercitiu-git, și vezi fișierul README.md.



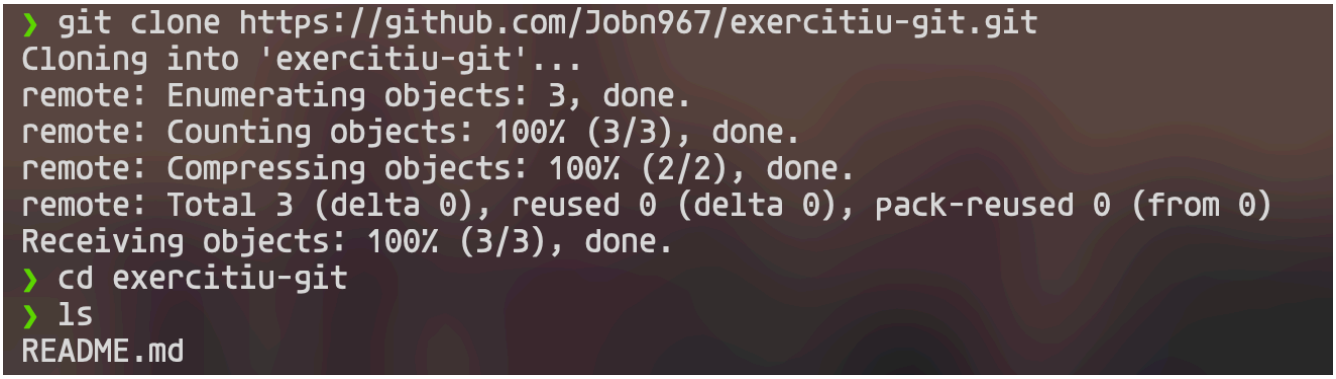
## A4. Îl aduci pe calculator: git clone (8 min)

Pe pagina depozitului apasă butonul verde Code, rămâi pe fila HTTPS și copiază adresa (se termină în .git). Apoi, în terminal, mergi într-un folder de lucru și clonează:

```
cd ~
git clone https://github.com/<numele-tau>/exercitiu-git.git
cd exercitiu-git
dir          # Linux: ls
```

**Pe ecran trebuie să vezi:**

```
Cloning into 'exercitiu-git'...
...
README.md
```




**De observat.** De acum lucrezi ÎN folderul exercitiu-git. Toate comenzile git de mai jos se dau de aici; dacă ieși din folder, git nu mai știe despre ce depozit vorbești.

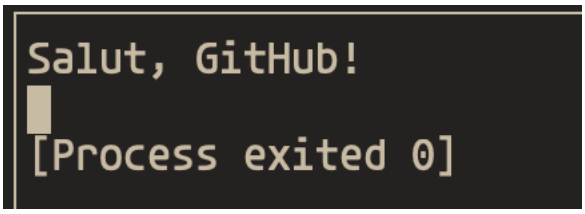
## A5. Prima ta fotografie: status, add, commit (12 min)

Creează un fișier nou, salut.py, din editor (nu cu echo din PowerShell, care scrie fișierul într-o codare pe care Python nu o citește). Scrie în el un singur rând, print("Salut, GitHub!"), salvează cu Ctrl+S, apoi rulează-l:

```
code salut.py        # sau deschizi folderul in editorul tau
python salut.py
```

**Pe ecran trebuie să vezi:**

```
Salut, GitHub!
```



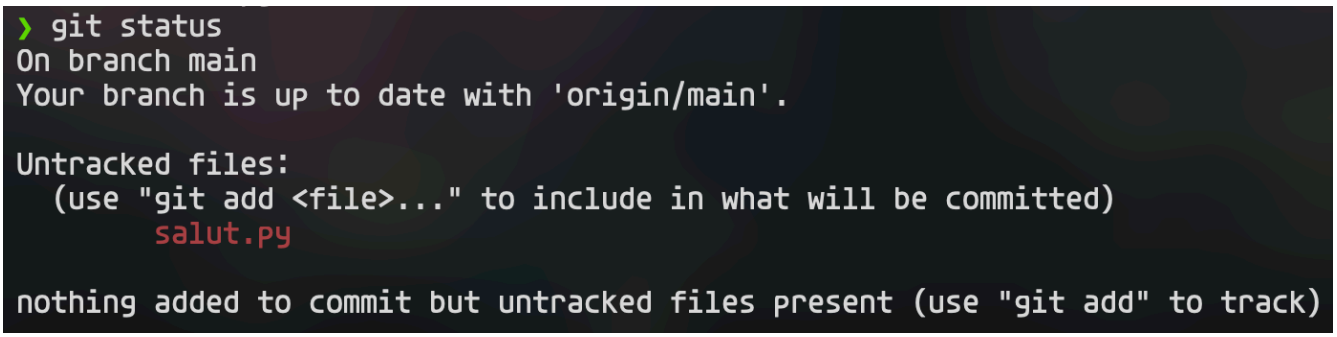

Întreabă git ce vede:

```
git status
```

**Pe ecran trebuie să vezi:**

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        salut.py
```

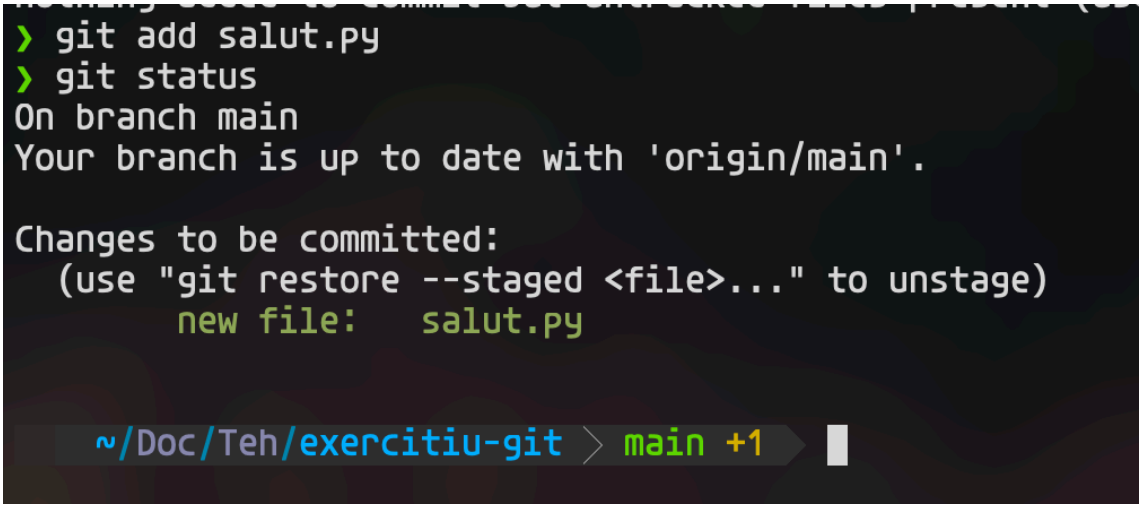



„Untracked" înseamnă: git vede fișierul, dar încă nu îl urmărește. Îl pui în fotografie cu add, apoi întrebi din nou:

```
git add salut.py
git status
```

**Pe ecran trebuie să vezi:**

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   salut.py
```

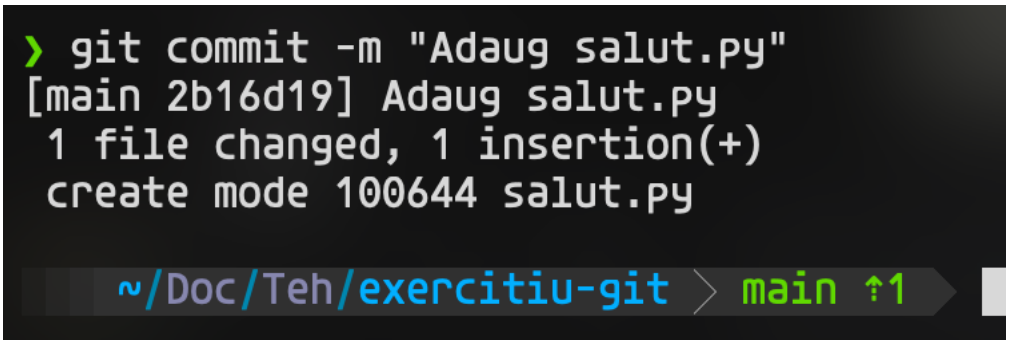



Fișierul e pregătit. Acum faci fotografia, cu un mesaj care spune ce ai schimbat:

```
git commit -m "Adaug salut.py"
```

**Pe ecran trebuie să vezi:**

```
[main aaf9ebc] Adaug salut.py
 1 file changed, 1 insertion(+)
 create mode 100644 salut.py
```




**De observat.** aaf9ebc e codul fotografiei; la tine e altul. Un avertisment de tipul „LF will be replaced by CRLF" e normal pe Windows și nu e o eroare.

## A6. O trimiți pe GitHub: git push (12 min)

```
git push
```

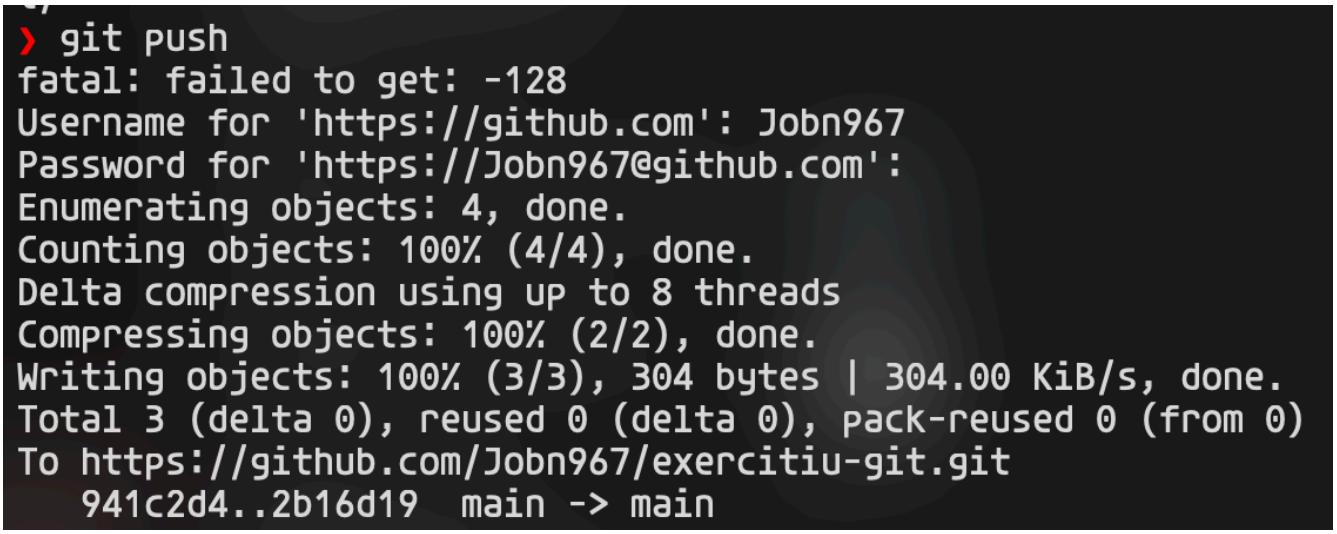
Prima dată, git îți cere să te autentifici la GitHub:

Pe Windows: se deschide o fereastră „Connect to GitHub". Alege Sign in with your browser, autentifică-te în browser și apasă Authorize. Fereastra se închide singură, iar push-ul continuă. Data viitoare nu te mai întreabă.

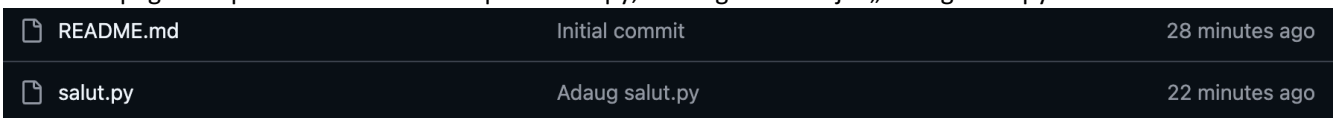
Pe Linux: când îți cere Username, scrii numele de utilizator GitHub. Când îți cere Password, NU merge parola contului: îți trebuie un token. Pe GitHub: poza ta (dreapta sus) → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic), bifezi repo, Generate, copiezi tokenul și îl lipești la Password. Păstrează-l ca pe o parolă.

**Pe ecran trebuie să vezi:**

```
To https://github.com/<numele-tau>/exercitiu-git.git
   e75c0d3..aaf9ebc  main -> main
```




Reîncarcă pagina depozitului în browser: apare salut.py, iar lângă el mesajul „Adaug salut.py".




## A7. O schimbare făcută pe GitHub, adusă înapoi: git pull (8 min)

În browser, deschide README.md, apasă creionul (Edit this file) și adaugă un rând, de exemplu: Aici exersez git. Jos apasă Commit changes, apoi din nou Commit changes. Acum GitHub are o fotografie pe care calculatorul tău nu o are. O aduci:

```
git pull
type README.md      # Linux: cat README.md
```

**Pe ecran trebuie să vezi:**

```
Updating aaf9ebc..1c2d3e4
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)

# exercitiu-git
Aici exersez gipt.
```

```
❯ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 990 bytes | 165.00 KiB/s, done.
From https://github.com/Jobn967/exercitiu-git
   2b16d19..2344110  main       -> origin/main
Updating 2b16d19..2344110
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
❯ cat README.md
# exercitiu-git
Exercitii de git pentru Tehnici de programare
Aici exersez git
```

**Regula de aur.** Înainte să începi lucrul, git pull. După ce ai terminat, git push. Așa calculatorul și GitHub rămân la fel.

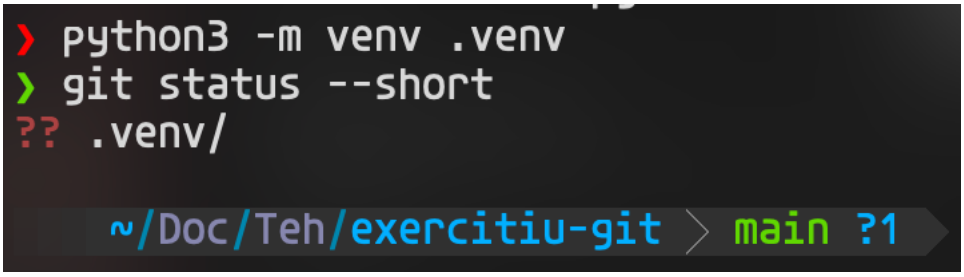
### A8. Ce NU urcă pe GitHub: .gitignore (8 min)

Mediul virtual (.venv) are mii de fișiere care nu au ce căuta în depozit. Creează-l și vezi ce crede git:

```
python -m venv .venv
git status --short
```

**Pe ecran trebuie să vezi:**

```
?? .venv/
```



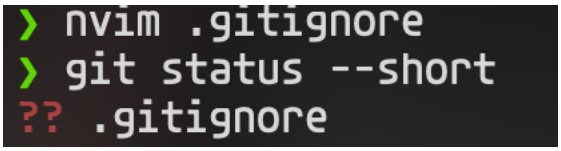

Dacă nu apare nimic, Python-ul tău (3.13 sau mai nou) a pus singur o regulă în .venv; faci oricum pasul, fiindcă .gitignore protejează și alte fișiere. Creează din editor un fișier numit exact .gitignore, cu două rânduri, și întreabă din nou:

```
# continutul fisierului .gitignore:
.venv/
__pycache__/

git status --short
```

**Pe ecran trebuie să vezi:**

```
?? .gitignore
```




.venv a dispărut din listă: git îl ignoră. Salvezi și trimiți regula:

```
git add .gitignore
git commit -m "Ignor mediul virtual"
git push
```

```
❯ git add .gitignore
❯ git commit -m "Ignor mediul virtual"
[main d3a4638] Ignor mediul virtual
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore
❯ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 336 bytes | 336.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Jobn967/exercitiu-git.git
   2344110..d3a4638  main -> main

~/Doc/Teh/exercitiu-git  main                    ✔  23s
```

## A9. Citești istoria și schimbările: log și diff (8 min)

```
git log --oneline
```

**Pe ecran trebuie să vezi:**

```
4f5a6b7 Ignor mediul virtual
1c2d3e4 Update README.md
aaf9ebc Adaug salut.py
e75c0d3 Initial commit
```

```
d3a4638 (HEAD -> main, origin/main, origin/HEAD) Ignor mediul virtual
2344110 Update README.md
2b16d19 Adaug salut.py
941c2d4 Initial commit
(END)
```

Adaugă un al doilea rând în salut.py, de exemplu print("al doilea"), și întreabă ce s-a schimbat, ÎNAINTE de commit:

```
git diff
```

**Pe ecran trebuie să vezi:**

```
--- a/salut.py
+++ b/salut.py
@@ -1 +1,2 @@
 print("Salut, GitHub!")
+print("al doilea")
```

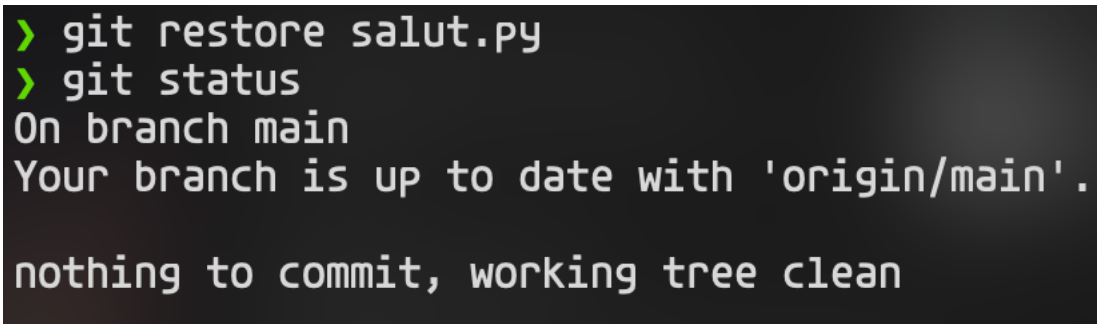
```
diff --git a/salut.py b/salut.py
index e700475..965afcb 100644
--- a/salut.py
+++ b/salut.py
@@ -1 +1,2 @@
 print("Salut, GitHub!")
+print("Rândul 2")
(END)
```

Rândul cu + e adăugat, un rând cu - ar fi fost șters. Te-ai răzgândit? Întorci fișierul la ultima fotografie:

```
git restore salut.py
git status
```

**Pe ecran trebuie să vezi:**

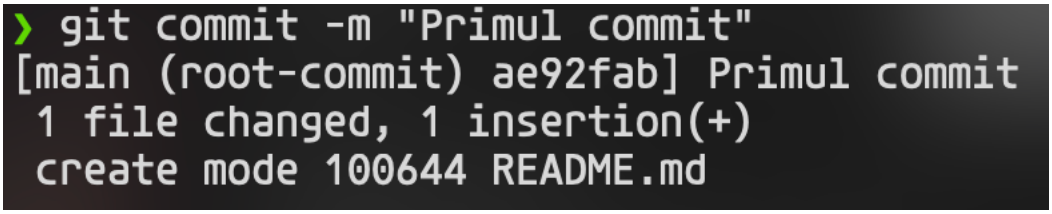
```
nothing to commit, working tree clean
```




### A10. Al doilea depozit, pornit de pe calculator (10 min)

De data asta pornești invers: întâi folderul local, apoi depozitul de pe GitHub. Ieși din exercitiu-git și creează un folder nou:

```
cd ~
mkdir exercitiu-2
cd exercitiu-2
git init
code README.md       # scrii un rand: # Al doilea depozit, si salvezi
git add README.md
git commit -m "Primul commit"
```

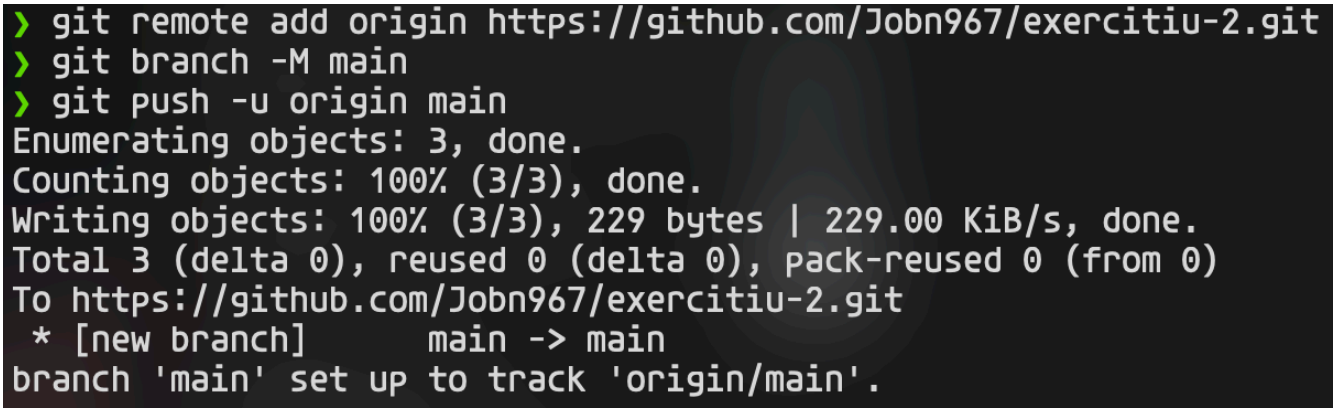



Pe GitHub creează un depozit nou exercitiu-2, Public, dar de data asta NU bifa Add a README file. GitHub îți arată o pagină cu instrucțiuni; folosești secțiunea „push an existing repository":

```
git remote add origin https://github.com/<numele-tau>/exercitiu-2.git
git branch -M main
git push -u origin main
```

**Pe ecran trebuie să vezi:**

```
branch 'main' set up to track 'origin/main'.
```

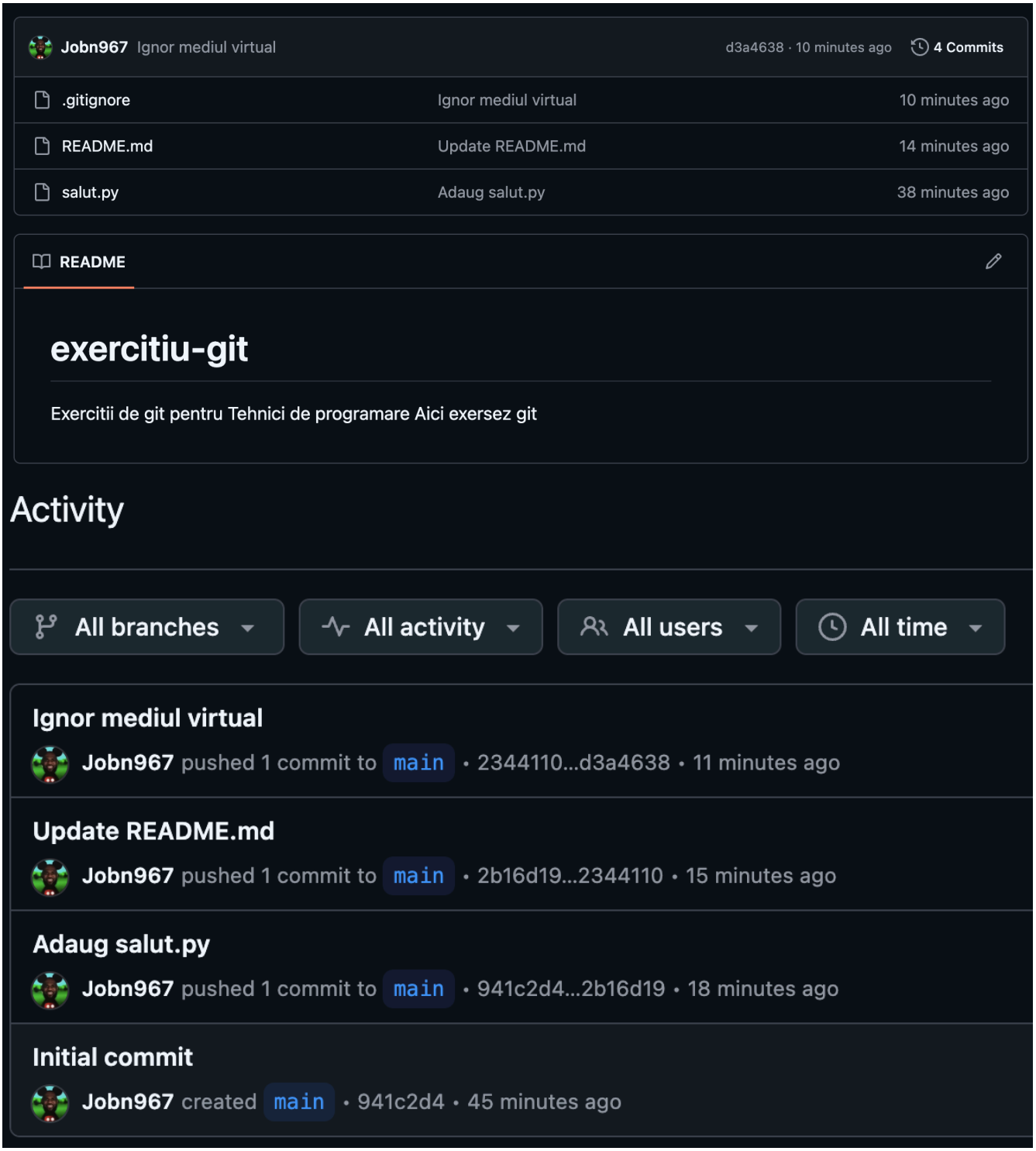


**De observat.** -u leagă ramura ta de cea de pe GitHub o singură dată; de acum încolo îți ajunge `git push` simplu.

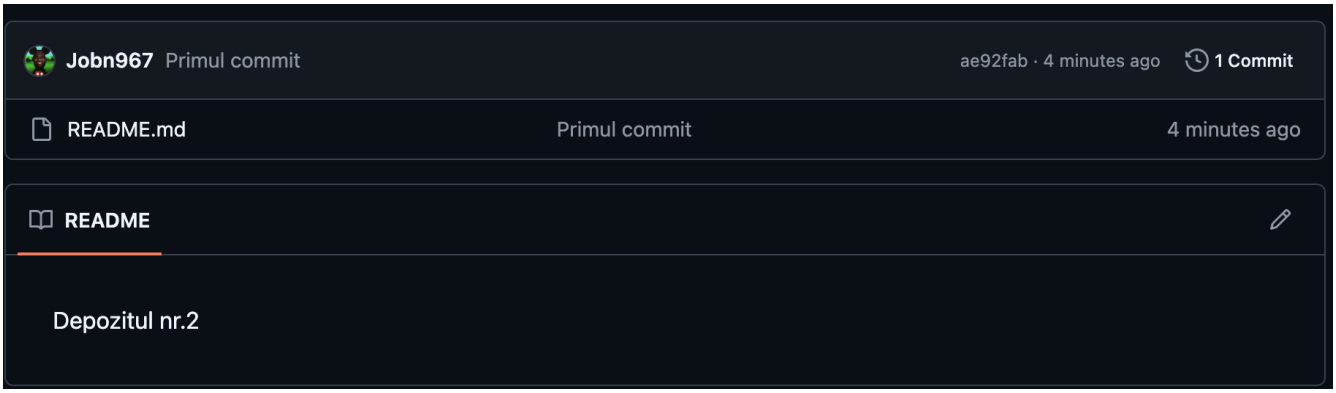
### A11. Verificarea finală (4 min)

Deschide în browser cele două depozite și verifică: exercitiu-git are README.md, salut.py și .gitignore, iar în istorie (linkul cu numărul de commits) cel puțin 4 commit-uri; exercitiu-2 are README.md. Ambele sunt Public.

**Repo-ul nr.1**




**Repo-ul nr.2**




### A12. Când ceva nu merge (0-10 min)

| Ce vezi | Ce înseamnă | Ce faci |
|---|---|---|
| `fatal: not a git repository` | nu ești în folderul depozitului | cd exercitiu-git, apoi repeți comanda |
| `Author identity unknown` | nu ai făcut A2 | rulezi cele două git config din A2 |
| `nothing to commit, working tree clean` | nu ai schimbat nimic de la ultimul commit | nu e o eroare: totul e salvat |
| `nothing added to commit but untracked files` | ai uitat git add | git add <fisier>, apoi commit |
| `! [rejected] main -> main (fetch first)` | GitHub are ceva ce tu nu ai | git pull, apoi git push |
| 403 sau Permission denied | te-ai autentificat cu alt cont sau tokenul e greșit | refaci autentificarea de la A6 |
| `LF will be replaced by CRLF` | avertisment despre sfârșitul de rând | normal pe Windows, îl ignori |

**Predai:** linkurile celor două depozite, `https://github.com/<numele-tau>/exercitiu-git` și `https://github.com/<numele-tau>/exercitiu-2`, pe care le arăți asistentului la verificare.

## Ce predai și cum se notează

| Livrabil | Ce conține |
|---|---|
| `github.com/<nume>/exercitiu-git` | Public; README.md, salut.py, .gitignore; cel puțin 4 commit-uri (A3-A9). |
| `github.com/<nume>/exercitiu-2` | Public; pornit cu git init și trimis cu push -u (A10). |

La verificare, asistentul deschide linkurile, citește istoria și îți poate cere să explici oricare commit sau oricare reparație. Istoria se scrie cinstit: predai ce ți-a ieșit, cu sursele notate dacă ai cerut ajutor.

## Reguli de bază, predare și notare

Această secțiune se aplică Laboratorului 6 și completează pașii de lucru de mai sus.

| Regula | Ce ai de făcut |
|---|---|
| Lucrezi individual | Înțelegi fiecare comandă pe care o rulezi. Dacă folosești o sursă sau primești ajutor, notezi acest lucru cinstit în raport. |
| Păstrezi datele private | Nu publici parole, token-uri, chei API sau fișiere cu date personale. Depozitele exercitiu-git și exercitiu-2 rămân publice doar fiindcă sunt exerciții fără date sensibile. |
| Ai un depozit privat pentru disciplină | Lucrările cursului se păstrează în TP_Nume_Prenume_Grupa și îl adaugi pe cristiangonceari-dotcom ca colaborator. |
| Folosești denumirea corectă | Raportul se numește exact L6_TP_Nume_Prenume_Grupa.md. Exemplu: L6_TP_Popescu_Ion_SI-266.md. |
| Salvezi regulat | Înainte de lucru rulezi git pull, iar după lucru faci git add, git commit și git push. Mesajele commit-urilor descriu pe scurt schimbarea făcută. |

### Ce predai

- În depozitul privat TP_Nume_Prenume_Grupa creezi folderul Laborator_6_GitHub.
- În folder pui raportul L6_TP_Nume_Prenume_Grupa.md, cu numele complet, grupa, username-ul GitHub, linkurile celor două depozite publice și câte o propoziție despre ce ai exersat la fiecare pas A1–A10.
- Depozitul exercitiu-git conține README.md, salut.py și .gitignore, are minimum 4 commit-uri și este Public.
- Depozitul exercitiu-2 conține README.md, este creat local cu git init, trimis prin push -u și este Public.
- După push trimiți un e-mail scurt la cristian.gonceari@isa.utm.md cu subiectul L6_TP_Nume_Prenume_Grupa și cu linkul depozitului privat plus codul ultimului commit.

### Barem: 10 puncte

| Criteriu | Verificare | Puncte |
|---|---|---|
| Configurare | Cont, git config și identitate configurate corect. | 2 |
| exercitiu-git | Fișierele cerute, .gitignore și minimum 4 commit-uri clare. | 3 |
| exercitiu-2 | Inițializat local și publicat corect pe GitHub. | 2 |
| Raport și predare | Raport denumit corect, linkuri funcționale și commit trimis în depozitul privat. | 1 |
| Înțelegere și onestitate | Poți explica comenzile de bază și istoricul propriilor commit-uri. | 2 |

**Nota lucrării este punctajul obținut, pe scala 1–10.**